Confidence in every change.

Proofline is a reusable testing harness for web apps built with React/Vite.

It brings automated checks, realistic user journeys, and release evidence into one repeatable workflow.

A harness coordinates how tests are configured, run, isolated, and checked as your application changes.

RULES

Does the result stay correct?

JOURNEYS

Can a user finish the task?

EVIDENCE

Can we explain the result?

The payoff: agreed behavior becomes checks you can run again on every change.

The insurance CRM is the reference proof. The harness is designed to be adapted to other React/Vite applications.

Turn agreed behavior into lasting checks.

Once a behavior is captured in a test, each code change can be checked against it again.

01

Describe

Your rules, users, and important tasks.

02

Capture

Write deterministic automated checks.

03

Repeat

Run the checks as the code changes.

04

Explain

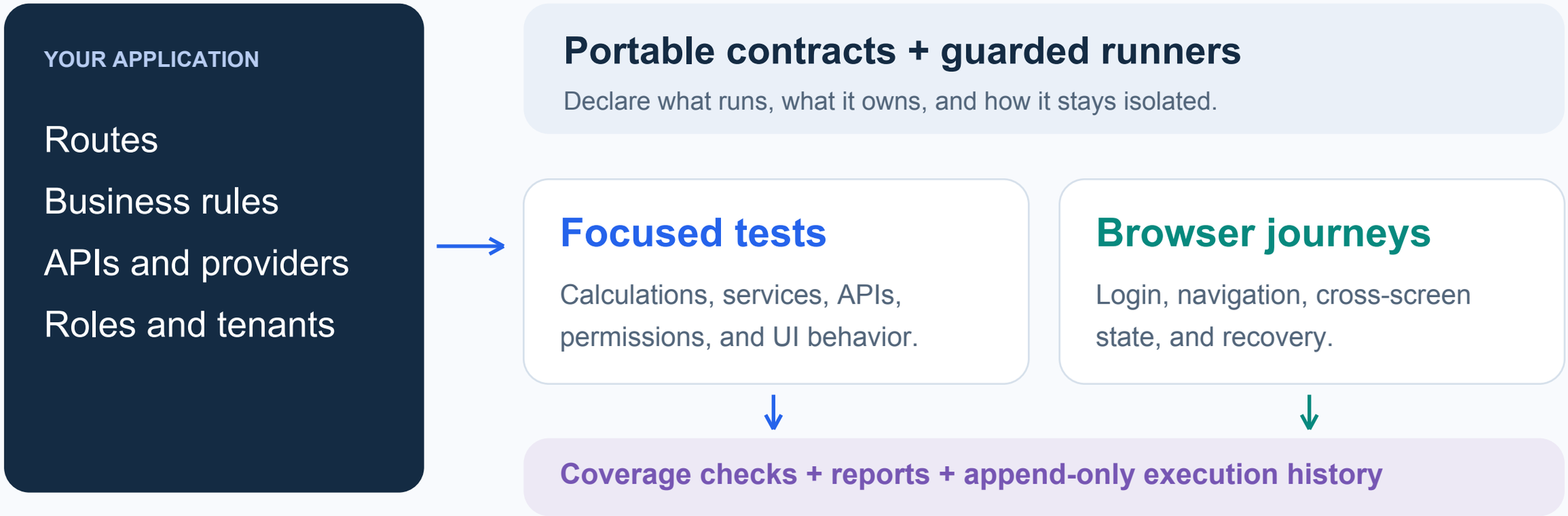
Keep results, reasons, and failure traces.

An agent can help author and adapt the tests. Continuous integration (CI) runs the saved tests using deterministic commands, with no model call in test execution.

Deterministic means the same controlled inputs produce the same expected result. The reference tests use local fakes rather than live providers.

One harness. Two kinds of proof.

Focused tests check individual behavior. Browser journeys check how the visible application works together.



The manifest is a checked-in configuration contract. Vitest runs focused tests; Playwright runs browser journeys.

Follow the journey a user actually takes.

The CRM reference proves a complete path through visible controls and navigation.



Sign in

Open a protected link; return after login.

Create

Add a customer, property, and quote.

Bind

Create a policy; inspect its detail.

Finance

Record a payment; see the ledger.

Sign out

Revisit the protected route and be denied.

Beyond a screenshot: the journey also checks that the policy, finance, and event state match the visible outcome.

Source: the executable golden workflow. Separate reference journeys cover response-loss recovery, access denial, tenant separation, and refund/correction.

Protect the behavior people depend on.

The reference implementation demonstrates checks across business behavior and testing boundaries.

Correct results

Rules, calculations, and API results stay correct.

Safe recovery

A retry does not create a second policy or duplicate ledger result.

Access boundaries

Roles and tenants cannot read or change data they do not own.

Independent tests

Fresh scenario state prevents one test from contaminating another.

Controlled requests

Undeclared requests fail; declared providers use deterministic fakes.

Production separation

Test-only adapters must be absent from the production build.

These checks prove declared scenarios and boundaries. Each new application needs its own rules, adapters, and tests.

Coverage is measured. Baselines cannot slip.

Coverage records which code tests execute. Approved minimums may rise; ordinary changes cannot lower them.



COVERED CODE LINES IN GOVERNED FILES, GROUPED BY COMPONENT

32

critical rules in the registry

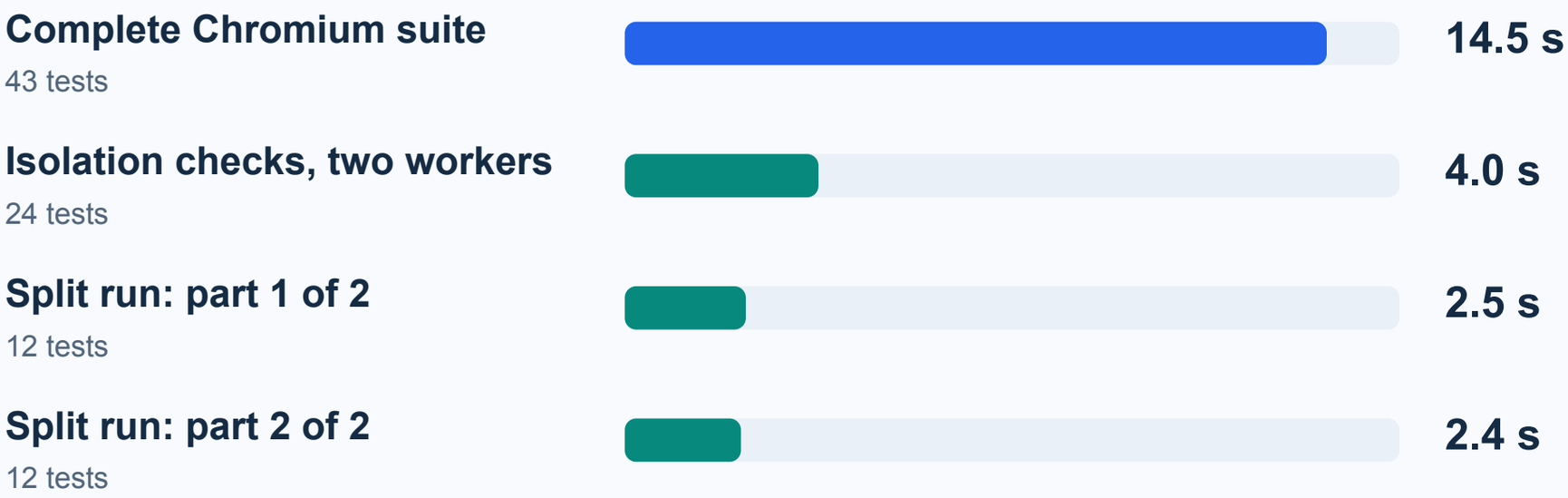
8 / 8

governed groups passed

Bars: covered lines / instrumented lines for the 12 classified source files, grouped by the manifest. Exclusions apply; this is not whole-application or browser coverage. The gate checks lines, branches, functions, and statements per file.

Repeatable checks, with visible results.

Two retained local gate passes provide a reference baseline for what ran.



404

unit tests
in 45 files

2

full local
gate passes

If a change cannot be mapped safely, selection expands to the full declared suites. Invalid execution metadata blocks the run.

Timings: canonical-full Pass 2 runner-reported wall time; builds excluded. Different workloads, not a speedup comparison. The 24 scenario tests are included in the 43-test suite; shards repeat those same scenarios. Local results vary by machine.

Bring your app. Reuse the testing discipline.

Proofline is a repository-native starting point for another React/Vite application.

Reuse the harness

Manifest + validators
Guarded execution
Change-aware diagnostics
Coverage + evidence policies

Adapt to your product

Routes + user journeys
Business rules + API contracts
Roles + tenant boundaries
Test data + provider fakes

Current scope: the complete declared browser suite runs in Chromium; Firefox/WebKit offer optional smoke checks. This is tooling in a repository. Adapting the contracts and tests requires engineering work.

Go deeper: TECHNICAL_GUIDE.md | diagrams, commands, measurements, source evidence